SHOPEASY QA

Jira Assignment — Complete Step-by-Step Implementation Guide

FROM PROJECT CREATION TO FINAL REPORT

Project: ShopEasy QA
Project key: SEQ
Platform: Jira Cloud
Approach: Scrum + manual test management using a Test issue type and Execution Status field

This edition explains not only what each assignment task means, but also the practical Jira navigation, fields to enter, issue relationships, workflow transitions, filter queries, gadget settings, and checks to perform.

A practical walkthrough and revision document based on the assignment brief and the work history available from the ShopEasy QA project.

How to Use This Guide

Follow Tasks 1–10 in order. Each task contains: objective, Jira navigation, actions, what to enter, and a completion check. Jira Cloud menus may differ slightly between company-managed and team-managed projects. If a menu is unavailable, check your project type and permissions instead of assuming the feature is missing.

Important distinction: This is a how-to record of the assignment workflow. Exact issue keys are generated by Jira and may differ. Use the keys shown in your own SEQ project when taking screenshots or submitting evidence.

Assignment requirements at a glance

Requirement	Minimum / target
Project	Scrum project named ShopEasy QA; key SEQ
Epics	At least 3
Stories	12–15; this project plan used 14
Sprints	2 one-week sprints; complete them genuinely
Test issues	At least 25; at least 10 positive, 10 negative, 5 edge/boundary
Traceability	Every requirement → story; every story → at least one test; every failed test → bug and story
Defects	At least 5; supplied observations identify 8 requirement mismatches
Bug workflow	Open → In Progress → Fixed → Retest → Closed; Retest → In Progress return
Filters / dashboard	At least 5 saved JQL filters; at least 4 working gadgets
Final report	1–2 page Test Summary Report with actual results and Go/No-Go recommendation

TASK 1 — Create the Jira Project

Objective

Create the workspace where all ShopEasy requirements, stories, tests, defects, sprints, filters, and dashboard items will live.

Step-by-step Jira actions

1. Sign in to your Jira Cloud site.

2. Open Projects from the top navigation and choose Create project.

3. Choose the Scrum template. Select the company-managed or team-managed option according to the assignment environment and available features.

4. Enter project name: ShopEasy QA.

5. Enter project key: SEQ. Check that Jira accepts it before creating the project.

6. Create the project and open its Backlog or board.

7. Open Project settings. Review the issue types available in the project.

8. Confirm that Epic, Story, Task, Bug, and Subtask are available.

9. Because this implementation uses Jira without Xray or Zephyr, create a custom issue type named Test if the project does not already have it. Open Jira settings → Issues → Issue types (navigation may vary), choose Add issue type, name it Test, select a suitable standard/standard-like type, and add it to the project's issue type scheme.

10. Return to the ShopEasy QA project and confirm that Test is available in the Create issue type selector.

What you should see

```
Project: ShopEasy QA
Project key: SEQ
Issue types: Epic, Story, Task, Bug, Subtask, Test
```

Completion check: the ShopEasy QA project opens, the board and backlog are available, and you can create the required issue types.

TASK 2 — Create Epics and User Stories

Objective

Convert the 16 business requirements (REQ-A1 to REQ-C6) into three feature groups and 14 user stories. This creates traceability from the business requirement to development and testing.

Step 1: Create the three epics

1. Open the ShopEasy QA project and select Backlog or the project issue list.
2. Select Create. Choose issue type Epic.
3. Create the first epic: Account and Login.
4. Repeat for Browse and Search.
5. Repeat for Cart and Checkout.
6. Open each epic and confirm the summary and description are meaningful.

Epic	Requirement coverage
Account and Login	REQ-A1, REQ-A2, REQ-A3, REQ-A4, REQ-A5
Browse and Search	REQ-B1, REQ-B2, REQ-B3, REQ-B4, REQ-B5
Cart and Checkout	REQ-C1, REQ-C2, REQ-C3, REQ-C4, REQ-C5, REQ-C6

Step 2: Create 14 stories and place them under epics

For every story, select Create → Story, write the user story in the summary/description, add requirement IDs, add the matching requirement ID as a label, enter acceptance criteria, set story points, and associate it with the correct Epic using the Parent/Epic Link field available in your Jira layout.

Epic	Suggested story breakdown	Requirement IDs
Account and Login	Registration; Login; Account lock; Forgot password; Logout	REQ-A1 to REQ-A5
Browse and Search	Product listing and pagination; Search; Product filters; Sorting and product details	REQ-B1 to REQ-B5
Cart and Checkout	Cart count and add item; Quantity and line total; Remove item; Coupon; Checkout, payment and confirmation	REQ-C1 to REQ-C6

Step 3: Write each story properly

Use the user-story pattern:

```
As a [type of user], I want [goal], so that [benefit].
```

Example for REQ-A2:

```
As a registered customer, I want to log in with my email and password, so that I can access my account.
```

Add at least two acceptance criteria in Given / When / Then format:

```
1. Given I am on the login page, when I enter a valid registered email and password, then I land on the home page.
2. Given I am on the login page, when I enter invalid details, then I see 'Invalid email or password'.
```

Step 4: Add requirement traceability

1. Put the relevant requirement ID, such as REQ-A2, in the story description.
2. Add the same requirement ID as a label, for example req-a2 (use a consistent label format).
3. Enter story points using the allowed scale: 1, 2, 3, 5, or 8.
4. Associate the story with its correct Epic.
5. Save and reopen the story to verify the Epic, label, acceptance criteria, and points.

Completion check: at least 3 epics and 12–15 stories exist; every REQ-A1–REQ-C6 requirement is represented; every story has at least two acceptance criteria, a requirement label, story points, and an Epic parent.

TASK 3 — Plan and Run Two Sprints

Objective

Divide the 14 stories into two one-week sprints, prioritize risk, and use the Scrum board to record actual progress.

Step-by-step Jira actions

1. Open the project's Backlog.

2. Choose Create Sprint.

3. Create Sprint 1 and Sprint 2.

4. Drag or move the selected stories from the backlog into Sprint 1 and Sprint 2.

5. Open each story and confirm story points are assigned.

6. Plan Sprint 1 with seven stories and 29 story points; plan Sprint 2 with seven stories and 34 story points, as recorded in the working plan. If the live Jira board differs, use the actual board values for the final submission.

7. Open Sprint 1 settings, enter a one-week date range and a clear Sprint Goal, then select Start Sprint.

8. During the sprint, move cards across the board as the work is performed. Do not mark work complete unless the task was actually completed.

9. Complete Sprint 1. Review any incomplete issues and move them back to the backlog or into Sprint 2 according to the actual outcome.

10. Start Sprint 2, update its cards as work progresses, and complete it after the work is finished.

Sprint allocation recorded in the project plan

Sprint	Stories	Story points
Sprint 1	7	29
Sprint 2	7	34

Completion check: both sprints appear in the board's completed sprint history. A burndown chart needs real sprint activity; do not fabricate status changes or dates just to create a chart.

TASK 4 — Create Test Cases

Objective

Create at least 25 Jira issues of type Test. The project test pack planned 32 test issues. Tests must cover every requirement and include positive, negative, and edge/boundary scenarios.

Step-by-step: create one Test issue

1. Open the ShopEasy QA project and select Create.

2. Choose issue type Test.

3. Enter a clear summary, such as Verify login rejects an incorrect password.

4. Enter preconditions, numbered steps, test data, and expected result in the description.

5. Set priority and labels. Use consistent labels such as positive, negative, or edge.

6. Link the Test issue to the relevant Story using the configured issue relationship (the assignment allows tests or relates to).

7. Create the issue and verify the link by opening the Test issue and checking Linked issues.

8. Repeat until the minimum test count and coverage mix are achieved.

Required test mix

Category	Minimum	Example
Positive	10	Register with valid email and a password meeting all rules.
Negative	10	Register with a password missing a special character.
Edge / Boundary	5	Quantity 1, 10 and 11; coupon threshold Rs. 999 and Rs. 1,000.

Example Test issue

Field	Value
Summary	Verify account locks after five consecutive failed login attempts
Preconditions	Registered user exists and is not locked.
Steps	1. Open login page. 2. Enter registered email and wrong password. 3. Repeat five consecutive times.
Expected result	After the fifth failed attempt, account is locked for 15 minutes.
Priority / labels	High / negative / req-a3
Link	Link to the Account Lock story.

Coverage rule

Every requirement must have at least one positive test and at least one negative or edge test. Pay particular attention to numeric boundaries: password length 8, five failed attempts, 15-minute lock, 30-minute reset link, quantity 1–10, and coupon minimum Rs. 1,000.

Completion check: at least 25 Test issues exist (the project plan targeted 32), the required category mix is met, and each test is linked to a story.

TASK 5 — Create Test Plan and Execute Tests

Objective

Record the testing scope and approach, create an Execution Status field for Test issues, and set test outcomes using the eight supplied observations.

Step 1: Create the Execution Status custom field

1. Open Jira settings → Issues → Custom fields (menu wording may vary).
2. Choose Create custom field.
3. Select a single-select list field.
4. Name it exactly Execution Status.
5. Add options: Not Run, Pass, Fail, Blocked.
6. Associate the field with the Test issue type and the relevant project/screens or field context.
7. Open a Test issue and verify that Execution Status is visible and can be edited.

Step 2: Create the test plan Task

1. Select Create and choose issue type Task.
2. Use the title Test Plan - ShopEasy 1.0 (the working issue was later referred to as SEQ-45 — ShopEasy QA - Test Plan and Execution).
3. Add the scope: Account and Login, Browse and Search, Cart and Checkout.
4. Add approach: requirement-based manual testing using the supplied observations.
5. Add environment: ShopEasy build 1.0.3, staging server, Chrome latest, Windows 11.
6. Add schedule: two one-week sprints / assignment period.

7. Add risks: security controls, search and sorting, quantity and coupon validation, and order confirmation.

8. Create the task and verify its title, description, and status.

Step 3: Execute tests against the given observations

Compare what happened with the exact requirement. Mark the related test Fail only when the observation violates the requirement. The assignment states that tests not related to any observation are treated as Pass.

Observation	Requirement	Observed result and decision	Related test
OBS-1	REQ-A1	Password abc12345 has no special character but registration succeeds — FAIL.	TC-02
OBS-2	REQ-A3	Account unlocks after 5 minutes, but must remain locked for 15 minutes — FAIL.	TC-06 / TC-07
OBS-3	REQ-A4	Reset link works after 45 minutes, but validity is only 30 minutes — FAIL.	TC-09
OBS-4	REQ-B2	Search is case-sensitive; 'SHIRT' fails although 'shirt' finds product — FAIL.	TC-14
OBS-5	REQ-B4	High-to-low result is not descending — FAIL.	TC-18
OBS-6	REQ-C2	Quantity 11 is accepted although maximum is 10 — FAIL.	TC-23
OBS-7	REQ-C4	SAVE10 gives discount on Rs. 300 although minimum cart value is Rs. 1,000 — FAIL.	TC-27
OBS-8	REQ-C6	UPI confirmation page omits unique Order ID — FAIL.	TC-31

Step 4: Update each Test issue

1. Search for the Test issue key, such as the test associated with TC-02.

2. Open the issue and edit Execution Status to Fail for the failed observations.

3. Add a comment explaining the observation and the requirement ID.

4. Set tests with no related failing observation to Pass, as directed by the assignment.

5. Use Blocked only if execution genuinely cannot proceed; do not use it as a substitute for Fail or Not Run.

6. Save and verify the field value on the issue.

Completion check: every Test issue has an Execution Status; each failure has an explanation and will be linked to a Bug in Task 6.

TASK 6 — Log Defects and Link Them

Objective

Create one Bug for each confirmed observation that contradicts a requirement. The eight supplied observations are all identified as mismatches in the project guide.

Step-by-step: create a Bug

1. Select Create and choose issue type Bug in project SEQ.

2. Enter a short summary describing one defect only.

3. Enter Environment: Build 1.0.3, staging server, Chrome latest, Windows 11.

4. Write exact numbered reproduction steps based on the observation.

5. Write the expected result using the requirement ID and the requirement wording.

6. Write the actual result using the observation.

7. Choose Severity (Critical, Major, Minor, Trivial) and Priority (Highest, High, Medium, Low) based on impact and urgency.

8. Attach the relevant observation evidence from the assignment brief if available.

9. Create the Bug.

10. Open the Bug, use Link issue, choose the appropriate relationship, and link it to the failed Test and the related Story. Repeat for each failed test.

Observation	Defect summary	Expected vs actual
OBS-1	Registration accepts password without special character	REQ-A1 requires at least one number and one special character; abc12345 was accepted.
OBS-2	Account unlocks before 15 minutes	REQ-A3 requires 15-minute lock; account works after 5 minutes.
OBS-3	Password reset link remains valid after 30 minutes	REQ-A4 requires 30-minute validity and single use; link worked after 45 minutes.
OBS-4	Product search is case-sensitive	REQ-B2 requires case-insensitive search; 'SHIRT' returns no results.
OBS-5	Price high-to-low sorting is incorrect	REQ-B4 requires descending price order; observed order is 999, 450, 3,500, 1,200.
OBS-6	Cart accepts quantity above 10	REQ-C2 allows quantity 1–10; quantity 11 is accepted.
OBS-7	SAVE10 applies below minimum cart value	REQ-C4 requires cart value of at least Rs. 1,000; discount applied to Rs. 300.
OBS-8	UPI confirmation omits Order ID	REQ-C6 requires unique Order ID; confirmation page shows no ID.

Severity and priority decision

Severity measures impact; priority measures urgency. The project pack suggests Major severity and High priority as starting values for the eight observations, but the assignment expects the tester to reason about them. Review your actual Jira values and the trainer's scale before submission.

Linking correctly

```
Bug ← linked to → Failed Test
Bug ← linked to → Related Story
```

Use the issue's More / Link issue action, select the link type available in your Jira project (for example relates to), search for the issue key, and confirm. Verify both links on the Bug and the Test.

Completion check: at least five Bugs exist; the project work history records OBS-1 through OBS-8 as the eight planned/created defects; each bug is linked to its failed Test and the corresponding Story.

TASK 7 — Customize the Bug Workflow

Objective

Create the QA bug life cycle and demonstrate both successful retesting and a failed retest.

Required statuses

Status	Meaning	Responsible role
Open	Bug has been reported.	Tester
In Progress	Developer is fixing it.	Developer
Fixed	Developer says the fix is done.	Developer
Retest	Tester is verifying the fix.	Tester
Closed	Fix is verified and defect is gone.	Tester

Step 1: Open workflow configuration

1. Open Project settings.

2. Open Workflows. In some Jira project types, workflow editing is done through Jira settings or via board-column/status configuration.

3. Locate the workflow associated with the Bug issue type.

4. Edit or create a dedicated Bug workflow. If using a shared workflow, check the issue types that will be affected before changing it.

Step 2: Add statuses and transitions

1. Ensure Open and In Progress exist.

2. Add Fixed and Retest if they are missing.

3. Ensure Closed exists.

4. Connect the forward transitions: Open → In Progress → Fixed → Retest → Closed.

5. Add the required failure transition: Retest → In Progress.

6. Publish the draft workflow or complete the migration step if Jira asks for it.

7. Confirm the Bug issue type is mapped to the intended workflow through the workflow scheme.

```
OPEN → IN PROGRESS → FIXED → RETEST → CLOSED
RETEST → IN PROGRESS (failed retest)
```

Step 3: Move bugs through the workflow

1. Open one Bug and transition it from Open to In Progress.

2. Move it to Fixed to represent the developer's fix.

3. Move it to Retest for tester verification.

4. For a successful retest, move it to Closed.

5. For at least one failed retest, move a Bug from Retest back to In Progress.

6. Add a comment explaining why the fix failed, for example: 'Retest failed: the account becomes usable after 5 minutes; REQ-A3 requires a 15-minute lock.'

7. Verify the issue history shows the transitions and comment.

Completion check: workflow contains all five required statuses, the Retest → In Progress path exists, and at least one Bug history proves a failed retest with an explanatory comment. The assignment work recorded the workflow as created and a selected Bug moved to In Progress.

TASK 8 — Build the ShopEasy QA Dashboard

Objective

Create a dashboard named ShopEasy QA Dashboard with at least four working gadgets and share it with view access.

Step-by-step dashboard creation

1. Open Dashboards from the Jira navigation.

2. Choose Create dashboard.

3. Enter the name ShopEasy QA Dashboard.

4. Set the description if desired and choose appropriate sharing permissions.

5. Create the dashboard.

6. Choose Add gadget and search for the required gadgets.

7. Add at least four gadgets, then configure each gadget and save.

8. Arrange the gadgets in a readable layout.

Recommended four required gadgets

Gadget	Configuration steps
Sprint Burndown	Select the SEQ Scrum board and Sprint 1 or Sprint 2. Save. The sprint needs genuine status/estimate history.
Pie Chart	Select a saved filter for project = SEQ AND issuetype = Bug. Group by Status or Priority.
Created vs Resolved Chart	Select the saved Bug filter. Choose an appropriate time period and save.
Filter Results	Select the saved filter for open high-priority bugs. Display Key, Summary, Priority, Status, and Assignee.

Optional fifth gadget: show test execution status using a Pie Chart grouped by the Execution Status custom field, or a test-management app gadget if installed.

Sharing the dashboard

1. Open dashboard options or Edit dashboard.
2. Open the sharing/permissions configuration.
3. Grant View access to the intended trainer or group, according to the assignment instructions.
4. Save and test that the dashboard is visible to the intended viewer.

Completion check: dashboard name is ShopEasy QA Dashboard; at least four gadgets render useful data; dashboard is shared with view access. The work session recorded four gadgets added and dashboard work confirmed done.

TASK 9 — Create and Save JQL Filters

Objective

Save at least five reusable JQL filters. Always run each query first and verify that the results match the filter name.

How to create a saved filter

1. Open Filters or Issues → Search for issues.
2. Switch to Advanced search / JQL.
3. Enter one query below and run it.
4. Check the returned issues and confirm the project key is SEQ.
5. Select Save As and enter the exact filter name.
6. Repeat for at least five filters.
7. Open each saved filter to verify it still returns the intended issues.

Filter name	JQL
All open bugs	project = SEQ AND issuetype = Bug AND statusCategory != Done
Open high-priority bugs	project = SEQ AND issuetype = Bug AND priority in (Highest, High) AND status != Closed
Bugs by assignee	project = SEQ AND issuetype = Bug ORDER BY assignee
Failed tests	project = SEQ AND issuetype = Test AND "Execution Status" = Fail
Stories in Sprint 1	project = SEQ AND issuetype = Story AND sprint = "SEQ Sprint 1"
All project tests	project = SEQ AND issuetype = Test ORDER BY priority DESC
Bugs by severity	project = SEQ AND issuetype = Bug AND Severity = Major ORDER BY priority DESC

The assignment requires at least five filters. The final two are useful extras. If Jira does not recognize Execution Status or Severity, verify that the custom field exists and that its exact name is used. If the sprint name differs, copy the actual sprint name from the board.

Completion check: at least five filters are saved, named clearly, and return the correct issues. Share filters with the dashboard viewer if needed for gadget access.

TASK 10 — Prepare the Test Summary Report

Objective

Prepare a concise 1–2 page report for a manager explaining whether ShopEasy 1.0 is ready for release.

Information to collect from Jira

• Project name and key: ShopEasy QA / SEQ.

• Application and build: ShopEasy 1.0 / build 1.0.3.

• Environment: staging server, Chrome latest, Windows 11.

• Scope: REQ-A1–REQ-A5, REQ-B1–REQ-B5, REQ-C1–REQ-C6.

- Actual counts of epics, stories, completed sprints, and Test issues.
- Execution counts: Total, Passed, Failed, Blocked, Not Run.
- Defect counts grouped by severity and current status.
- Dashboard screenshot and workflow screenshot.
- Risks, observations, and a justified Go / No-Go recommendation.

Metrics to calculate

Metric	Formula
Pass percentage	Passed ÷ (Passed + Failed) × 100
Fail percentage	Failed ÷ (Passed + Failed) × 100
Defect density	Total defects ÷ number of stories (or test cases)
Requirement coverage	Requirements with at least one linked test ÷ total requirements × 100
Defect closure rate	Closed bugs ÷ total bugs × 100

How to complete the report accurately

1. Open the Test issues and use the Execution Status field or an export to count each status.
2. Open the Bug list and count defects by severity and status.
3. Check the Epics and Stories in the project to confirm final counts.
4. Check completed sprint history for sprint names and dates.
5. Capture a screenshot of the dashboard and the configured bug workflow.
6. Calculate metrics using actual values; do not estimate or copy sample numbers.
7. Write the executive summary and choose Go or No-Go based on the evidence and release criteria.
8. Export or save the final report as PDF or Word.

Important: the supplied observations show eight requirement mismatches. A No-Go recommendation is reasonable until the defects are fixed and retested, but the final recommendation must be based on the actual Jira execution evidence and release criteria.

Cross-Cutting Concept — Requirement Traceability

The most important principle in the assignment is traceability. Every requirement must connect to a story; every story must have at least one test; every failed test must have a bug; and every bug must link back to its failed test and story.

```
REQUIREMENT (REQ-A2)
↓ covered by
USER STORY (Login)
↓ verified by
TEST (TC-xx)
↓ if failed, creates
BUG (OBS-x)
↓ fixed and verified through
BUG WORKFLOW
```

Traceability checks before submission

- Every requirement REQ-A1 through REQ-C6 appears in at least one story.
- Every story is linked to the correct Epic and has acceptance criteria, label, and story points.
- Every Test issue is linked to a Story.
- Every failed Test has a Bug linked to it.
- Every Bug is also linked to the relevant Story.
- Bug workflow history demonstrates the required statuses and failed-retest return.
- Dashboard gadgets use saved filters that the viewer can access.

Final Submission Checklist

Deliverable	What to submit / verify
Jira project link	Share ShopEasy QA project with trainer view access.
Dashboard screenshot	Show ShopEasy QA Dashboard and at least four working gadgets.
Bug workflow screenshot	Show Open, In Progress, Fixed, Retest, Closed and Retest → In Progress.
Test export	Export Test issues and Execution Status results to CSV or Excel.
Test Summary Report	Submit final 1–2 page PDF or Word report with actual counts and metrics.
Minimum project content	3+ Epics; 12–15 Stories; 2 completed sprints; 25+ tests; 5+ bugs; 4+ gadgets; 5+ saved filters.

What We Learned by Completing This Assignment

• How to convert business requirements into Epics and user stories.

• How to add acceptance criteria, labels, story points, and parent relationships.

• How to plan work in Scrum sprints and update the board.

• How to create test cases as Jira issues and link them to stories.

• How to record test execution using a custom single-select field.

• How to compare observations with requirements and decide whether they are defects.

• How to write reproducible bug reports and link defects to failed tests and stories.

• How to configure a bug workflow and demonstrate retesting, closure, and reopening.

• How to create JQL filters and use them as dashboard data sources.

• How to communicate testing status through a dashboard and test summary report.

The assignment is complete only when the Jira project itself contains the required records and evidence. This PDF explains the implementation steps; use the live Jira issue keys, screenshots, sprint history, and execution counts as the source of truth for final submission.